

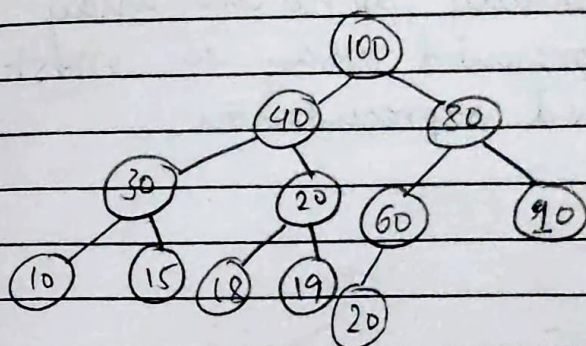
HEAP DATA STRUCTURE

Heap is a special type of complete binary tree used to store element in a specific order.

PROPERTIES OF HEAP

(i) Complete Binary Tree:

All levels are completely filled except possibly the last level. The last level is filled from left to right.

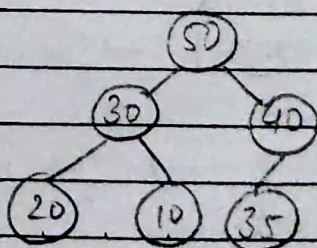


(ii) Two types of heap based on ordering :

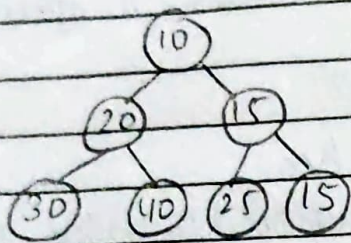
- ↳ max heap
- ↳ min heap

Max heap: The parent node is greater than or equal to its children.

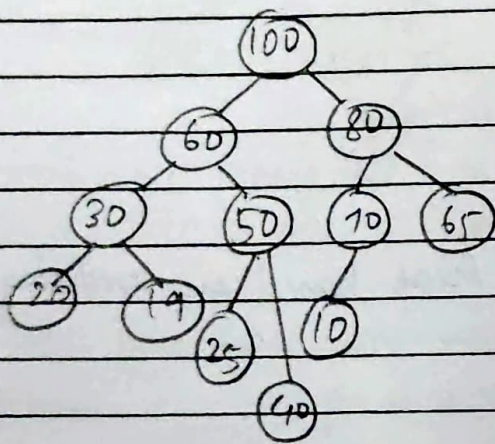
Parent $\geq$ children



Min heap: The parent node is smaller than or equal to its children.
 The smallest element is always at the root



(iii) Heap is usually stored in array
 Heap represented array is most than linked representation.



100	60	80	30	50	70	65	25	19	25	40	10
-----	----	----	----	----	----	----	----	----	----	----	----

if node is at index i :

left child = $2i + 1$

right child = $2i + 2$

parent node = $\frac{n}{2} - 1 \rightarrow$ for index (0)

= $\frac{n}{2} \rightarrow$ for index (1)

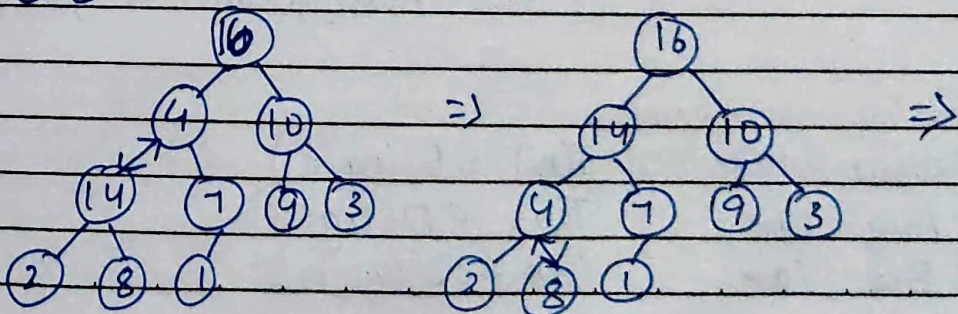
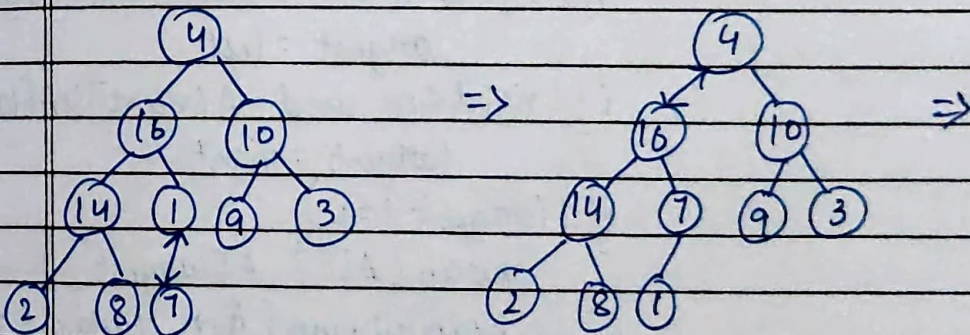
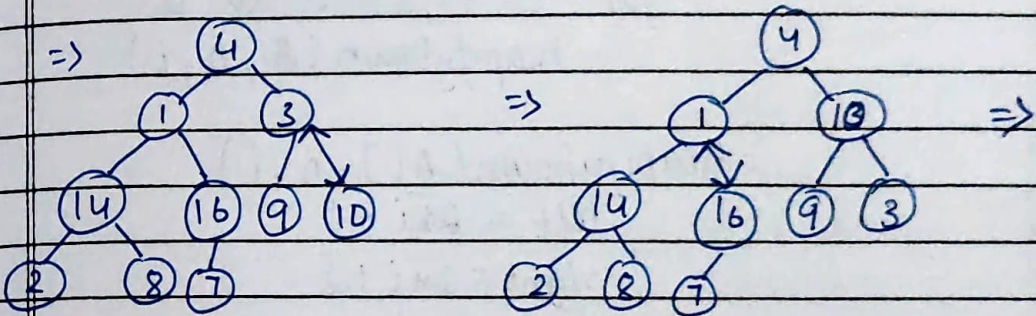
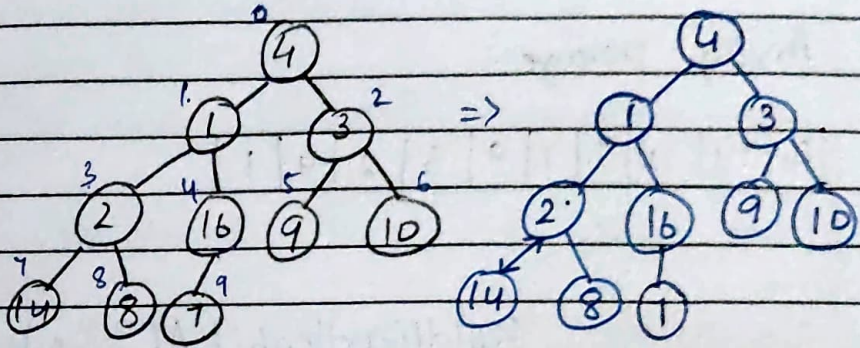
$$\text{left child}(i) = 2 \times 4 + 1 = 9$$

$$\text{right child}(i) = 2 \times 4 + 2 = 10$$

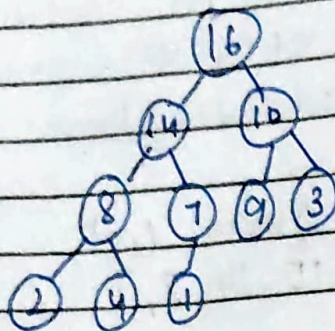
$$\text{parent}(i) = \frac{4}{2} - 1 = 2 - 1 = 1$$

Qn) Build max heap, given list
 4, 1, 3, 2, 16, 9, 10, 14, 8, 7

$$\frac{10}{2} - 1 = 5 - 1 = 4$$



=>



(Max heap).

Array storage:

16	14	10	8	7	9	3	2	4	1
----	----	----	---	---	---	---	---	---	---

ALGORITHM BuildMaxHeap ($A[]$, n)
 for $i = (n/2) - 1$ to 0
 heapifyDown (A , n , i)

heapifyDown ($A[]$, n , i)
 left = $2i + 1$
 right = $2i + 2$
 largest = i
 if (left < n and $A[\text{left}] > A[\text{largest}]$)
 largest = left
 if (right < n and $A[\text{right}] > A[\text{largest}]$)
 largest = right
 if (largest $\neq i$)
 swap ($A[i]$, $A[\text{largest}]$)
 heapifyDown (A , n , largest)

Time complexity

Worst case $T(n) \in \Theta(\log_2 n)$
 Avg case $T(n) \in O(\log_2 n)$
 Best case $T(n) \in \Omega(\log_2 n)$

HEAP SORT:

- Heap sort is a comparison based sorting algorithm that uses the Binary heap data structure
- Heap sort - Decrease and conquer technique

Time complexity

Best case $T(n) \in \Omega(n \log n)$

Worst case $T(n) \in O(n \log n)$

Avg case $T(n) \in \Theta(n \log n)$

Space complexity Auxiliary spaces $S(n) = O(1)$

ALGORITHM

Heapsort(A[1..n])

BuildMaxHeap(A, n)

for $i = n-1$ down to 1

swap(A[0], A[i])

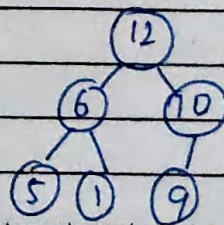
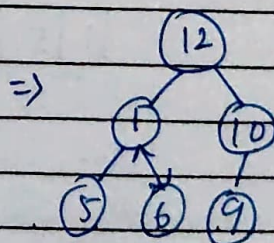
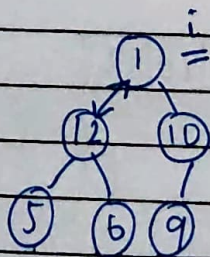
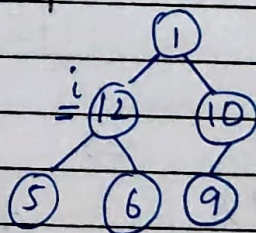
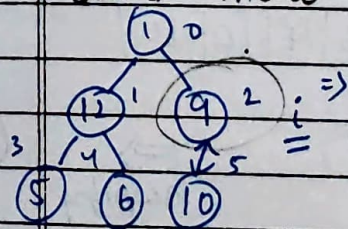
heapSize = heapSize - 1;

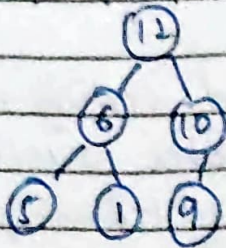
HeapifyDown(A, heapSize, 0)

Qn) Apply Heapsort for the given list

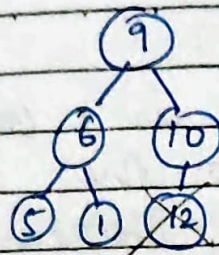
1, 12, 9, 5, 6, 10 (last) $i = \frac{n-1}{2}$

S.1 → Build max heap:

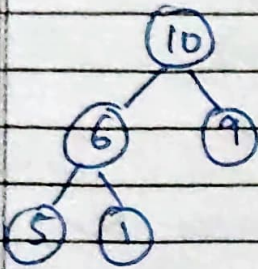
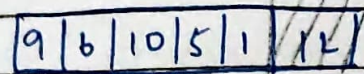
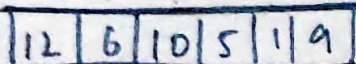




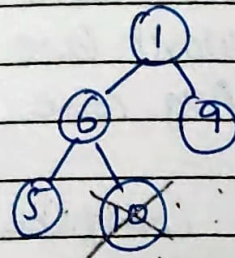
$\Rightarrow$
 swap(12, 9)



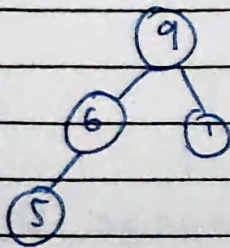
$\Rightarrow$
 swap(0)
 heapify



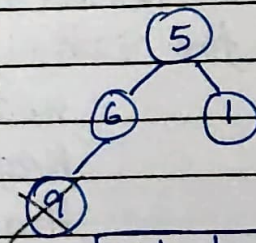
$\Rightarrow$
 swap(10, 1)



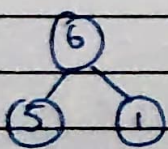
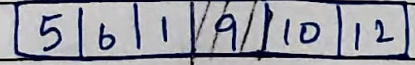
$\Rightarrow$
 swap(0)
 heapify



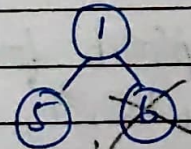
$\Rightarrow$
 swap(9, 5)



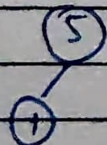
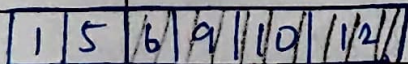
$\Rightarrow$
 heapify(0)



$\Rightarrow$
 swap(6, 1)



$\Rightarrow$
 heapify()

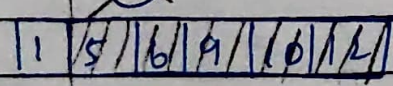


$\Rightarrow$
 swap(5, 1)

$\Rightarrow$



$\Rightarrow$
 heapify



23/10/24

// Time Complexity of Heap Sort

Basic operation, heapify tree

$$T(n) = \sum_{i=n-1} \text{heapifydown}(A, n, i)$$

Time complexity of heapify down tree = $O(n \log n)$

$$T(n) = \sum_{n-1}^1 \log n = (n-1) \log n$$

$n > > > 1$

$$\Rightarrow n \log n$$

PRIORITY QUEUE

- > Priority Queue is a Data Structure where each element has priority.
- > The element with highest priority is removed first

3	5	1	6	2	4
P ₁	P ₂	P ₃	P ₄	P ₅	P ₆

- > Priority Queue is implemented using **HEAP** for efficient operation.

Normal Priority queue $T(n) \in O(\log n)$

TYPE :

Maximum Priority Queue

- > Largest priority element removed first

Min priority Queue

- > Using binary heap

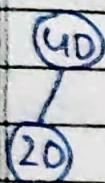
OPERATIONS:

- (i) insert $O(\log n)$
- (ii) delete $O(\log n)$
- (iii) peek $O(1)$

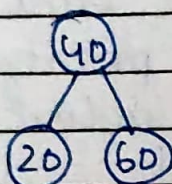
APPLICATION OF PRIORITY QUEUE

- > CPU scheduling
- > Network routing
- > Dijkstra Algorithm
- > Prim's algorithm
- > Heap Sort

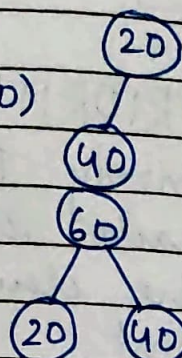
⇒ insert(20)



⇒ heapifyup



⇒ insert(40)

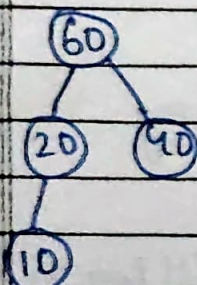


⇒ heapifyup

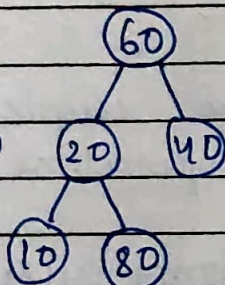
⇒ insert(60)

⇒ heapifyup()

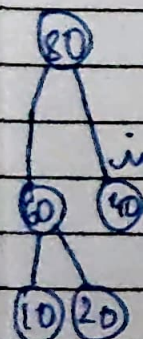
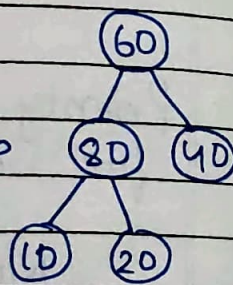
⇒ insert(10)



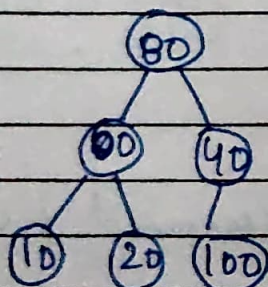
⇒ insert(80)



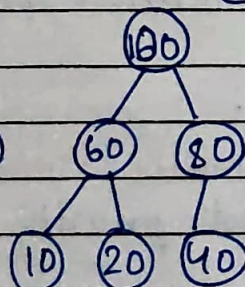
⇒ heapifyup



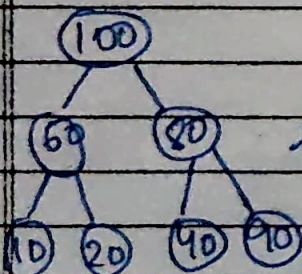
⇒ insert(100)



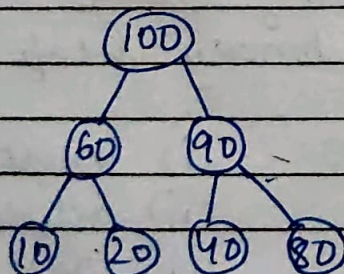
⇒ heapifyup()



⇒ insert(90)



⇒ heapifyup



PRIORITY QUEUE USING HEAP

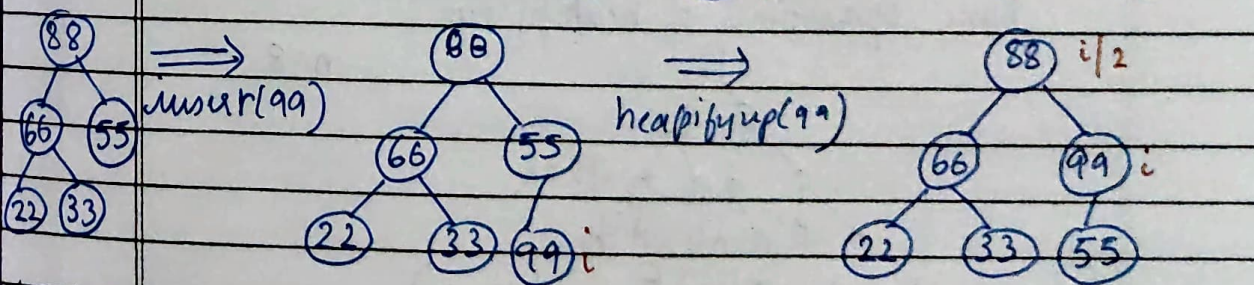
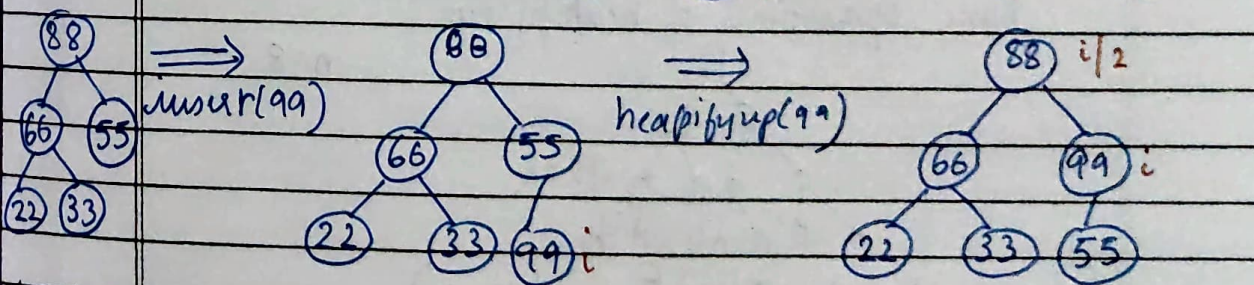
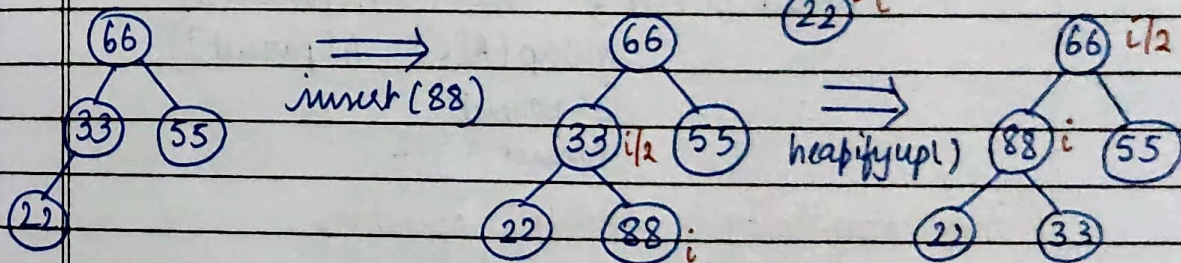
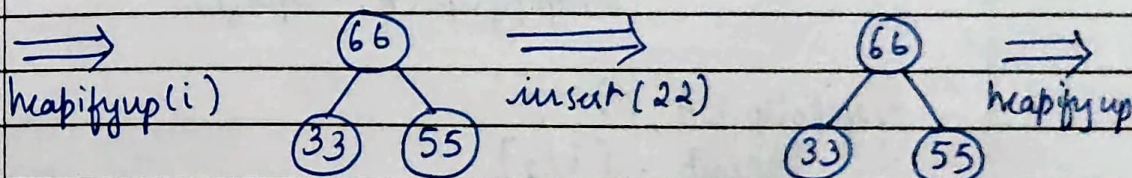
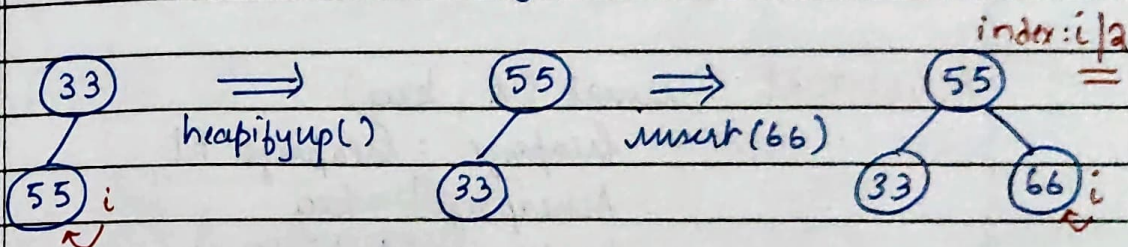
Operations:

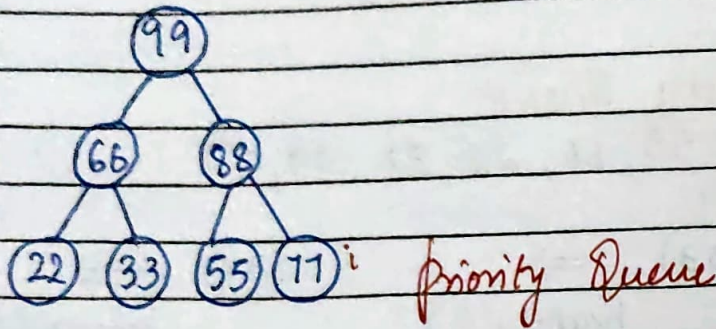
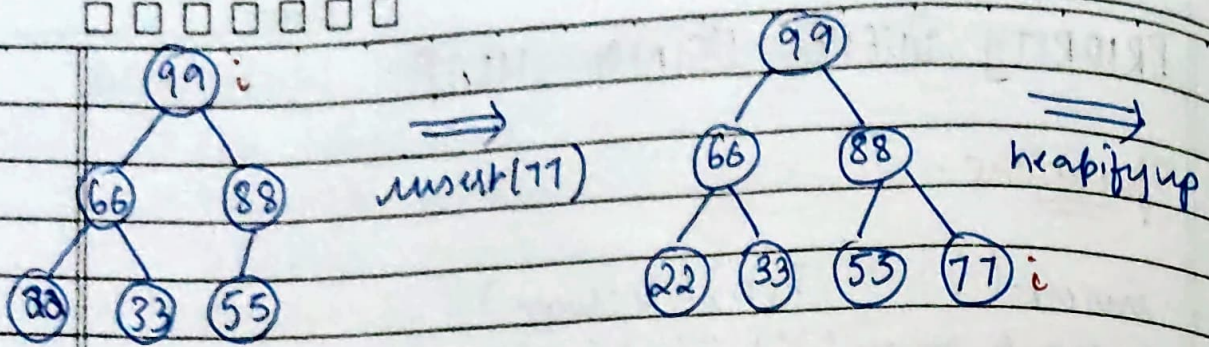
- (i) $insert()$: $T(n) \in O(\log n)$
- (ii) $Extract-max()$: $T(n) \in O(\log n)$
- (ii) $peek()$: $T(n) \in O(1)$

(i) $insert()$ PRIORITY QUEUE

- 33, 55, 66, 22, 88, 99, 17

$\Rightarrow$ $insert(33)$ $\Rightarrow$ $heapify()$ $\Rightarrow$ $insert(55)$

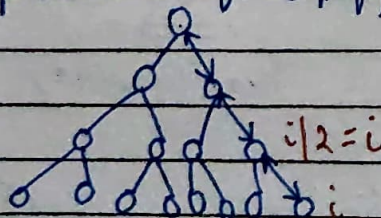




ALGORITHM $\text{insert}(A, \text{key})$
 $\text{heapsize} = \text{heapsize} + 1$
 $A[\text{heapsize}] = \text{key}$
 $\text{heapifyUp}(A, \text{heapsize})$

$\text{heapifyUp}(A, i)$
 $\text{parent} = \lfloor i/2 \rfloor$
 $\text{while } (i > 1 \text{ \& } A[i] > A[\text{parent}])$
 $\text{swap}(A[i], A[\text{parent}])$
 $i = \text{parent}$
 $\text{parent} = i/2$

// time complexity $\text{insert}()$ operation
 Basic operation of heapify up



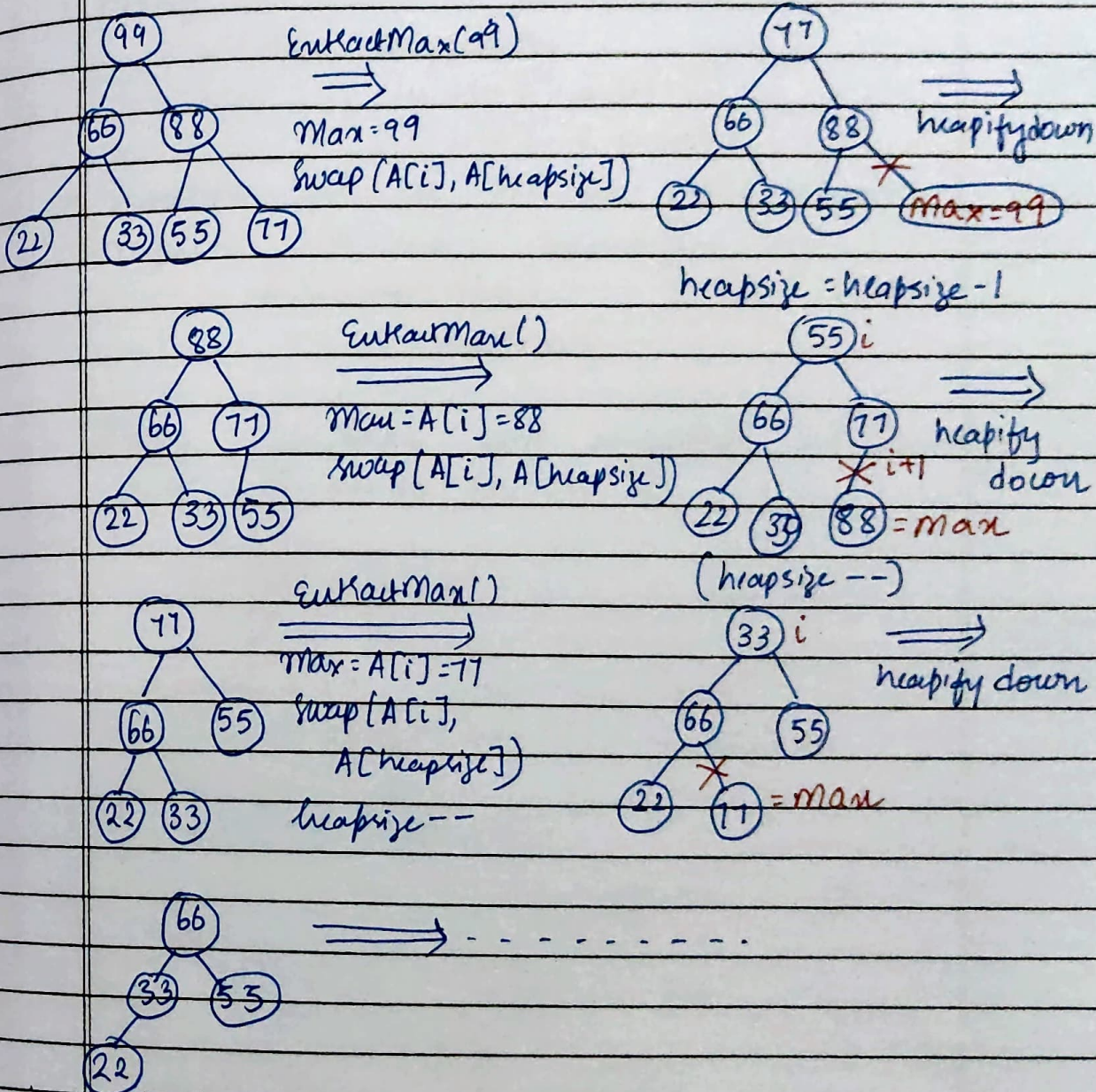
$\text{heapifyUp}()$ $T(n) \in O(\log_2 n)$

max moves by $i = 4$.
 $\log_2 15 \approx 4$

Time complexity $T(n) \in O(\log n)$

(2) `extractMax()` PRIORITY QUEUE

Purpose: to extract the largest element



M T W T F S S
☐ ☐ ☐ ☐ ☐ ☐ ☐

COMPASS
Date :

ALGORITHM

ExtractMax(A)

max = A[i]

heapSize = heapSize - 1

HeapifyDown(A, i)

return max

python built in sort()

intuition  Date: _____

M T W T F S S
☐ ☐ ☐ ☐ ☐ ☐ ☐

INSERTION SORT

- Insertion sort is to build by inserting each element into its correct position.
- Insertion sort based on Pearce and Langner technique
- Array is already sorted or almost sorted, insertion sort performs very fast.

Best case $T(n) \in \Omega(n)$

- Array is random order elements

Time Complexity

$$T(n) \in \Theta(n^2)$$

$$T(n) \in O(n^2)$$

- Insertion sort is an in-place algorithm and require no extra memory.

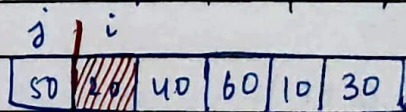
Auxiliary Space

$$S(n) \in O(1)$$

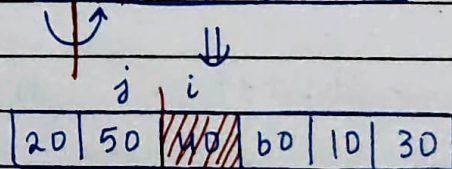
Complexity

Apply insertion sort algorithm for the given list

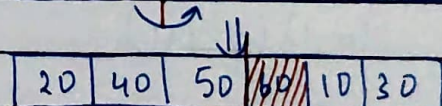
50, 20, 40, 60, 10, 30.



key = 20.

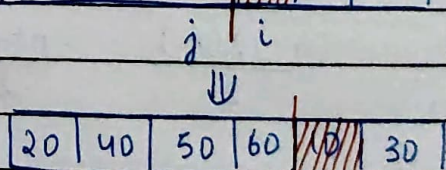


key = 40

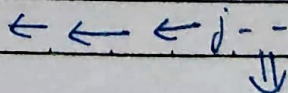


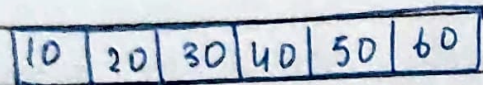
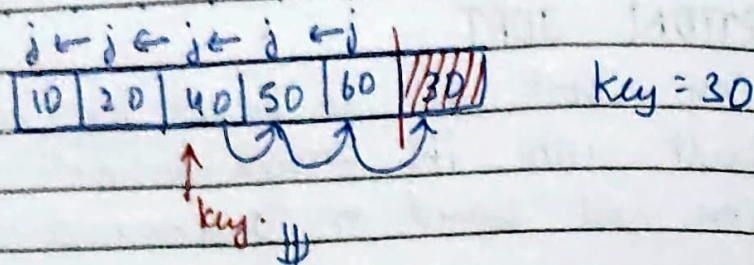
key = 60

$j \rightarrow j+1$ pointer
 $j--$ (Place back to prev index)



key = 10





ALGORITHM Insertion ($A[]$, n)
 for $i = 1$ to $n - 1$
 $key = A[i]$
 $j = i - 1$
 while ($j > 0$ and $key < A[j]$)
 $A[j + 1] = A[j]$
 $j = j - 1$
 end while
 $A[j + 1] = key$

Time complexity:

$$\begin{aligned}
 T(n) &= \sum_{i=1}^{n-1} \sum_{j=0}^{i-1} 1 \\
 T(n) &= \sum_{i=1}^{n-1} ((i-1) - 0 + 1) \\
 &= \sum_{i=1}^{n-1} i = 1 + 2 + 3 + \dots + n - 1 \\
 &= \frac{(n-1)(n-1+1)}{2} \\
 &= \frac{(n-1)(n)}{2} = \frac{n^2 - n}{2} \\
 T(n) &= \frac{n^2 - n}{2} \quad T(n) \in O(n^2)
 \end{aligned}$$

Best case Time complexity - Array Sorted list

- $key < A[j]$ is false in the first comparison
- while loop executes only once
- no shifting of element occurs

$$T(n) = \sum_{i=1}^{n-1} 1$$

$$= (n-1 - 1 + 1)$$

$$T(n) = n-1$$

$$\text{let } n \gg 1$$

$$T(n) \in \Omega(n) \Rightarrow \text{Best case}$$

COUNTING SORT

- Counting sort is a non-comparison sorting algorithm used to sort integer items when the range of values is known as small
- Counting sort is space trade off algorithm because it uses extra measures to reduce the sorting time.

//time complexity: (linear time complexity)

Best case $T(n) \in \Omega(n+k)$

Average case $T(n) \in \Theta(n+k)$

Worst case $T(n) \in O(n+k)$

$A[] = [2 | 5 | 3 | 0 | 2 | 3 | 0 | 3]$ input array
 $\Downarrow$ size = n

0 1 2 3 4 5 (k) maximum range = 5

$C[] = [0 | 0 | 0 | 0 | 0 | 0]$

initialize to zero
 (extra buffer)

Counting frequency of each element

	0	1	2	3	4	5
C[]	2	0	2	3	0	1

how many times 0 appears how many times 1 appears

Counts cumulative frequency

	0	1	2	3	4	5
C[]	2	2	4	7	7	8

→ Input array

	0	1	2	3	4	5
C[]	2	2	4	7	7	8

	1	2	3	4	5	6	7	8
A[]	2	5	3	0	2	3	0	3

→ Output array

	0	1	2	3	4	5	6	7	8
B[]	0	0	2	2	3	3	3	3	5

copy back to original array A[]

A[]	0	0	2	2	3	3	3	5
-----	---	---	---	---	---	---	---	---

↳ sorted list.

ALGORITHM Counting Sort

Counting Sort(A[], n, k)

let create c[0:k] & B[1:n]

for i = 0 to k
 c[i] = 0

for i = 1 to n
 c[A[i]] = c[A[i]] + 1

for i = 1 to k
 c[i] = c[i] + c[i-1]

```

for j = n to 0
    B[c[A[j]]]
    c[A[j]] = c[A[j]] - 1
return B
    
```

Time complexity

$$T(n) = \sum_{i=1}^n 1 + \sum_{i=0}^k 1 + \sum_{i=1}^n 1 + \sum_{i=1}^k 1 + \sum_{j=n}^0 1 + \sum_{j=1}^n 1$$

$$T(n) = n + (k+1) + n + k + n + n$$

$$T(n) = 4n + 2k + 1$$

Given constant

$$T(n) = O(n+k) \rightarrow \text{Best / Worst / Avg.}$$

Space complexity

$$S(n) = O(n+k)$$

$\nwarrow \quad \nearrow \quad \nwarrow$
 $B[1:n] \quad \quad C[0:k]$
 space space

RADIX SORT

- A non comparison sorting algorithm that sorts numbers digit by digit from least significant digit to most significant digit.
- > Radix sort uses counting sort as a subroutine for sorting each digit.
- > Radix sort is space - tradeoff algorithm [Radix sort uses extra memory to reduce execution time]

Time complexity $T(n) \in O(d(n+k))$
 Space complexity $S(n) \in O(n+k)$

Key idea of Radix Sort

- S.1 Find the maximum number
- S.2 Start sorting by unit place digit
- S.3 Then ~~the~~ ten's place of digit
- S.4 Then 100's place of digit etc
- S.5 Continue until the largest digit place

$n \rightarrow$ no. of inputs

$k \rightarrow$ range of digits i.e 0-9
 $= \underline{\underline{9}}$

M T W T F S S
☐ ☐ ☐ ☐ ☐ ☐ ☐

Date: _____

Apply Counting sort / Radix sort to the given list
 329, 457, 657, 839, 436, 720, 355

3	2	9		1	2	0		1	2	0
4	5	7	Phase:1	3	5	5	Phase:2	3	2	9
6	5	7	⇒	4	3	6	⇒	4	3	6
8	3	9	Sort. with	4	5	7	Apply	8	3	9
4	3	6	place	6	5	7	Counting	3	5	5
7	2	0	digit	3	2	9	Sort for	4	5	7
3	5	5		8	3	9	third place	6	5	7
							digit.			

	3	2	9
	3	5	5
Phase:3	4	3	6
⇒	4	5	7
Apply	6	5	7
Counting sort	1	2	0
to 100's place	8	3	9

ALGORITHM Radix Sort

RadixSort (A, d)

for $i = 1$ to d

use a stable sorting array A on digit's
 (using counting sort)

A → Array of numbers

d → no. of digits in the largest number.

Stable sorting Algorithm used: **counting sort.**

// Time complexity:

Radix sort uses Counting Sort Algorithm

for $i=1$ to d
 Counting (A, n, d)

$$T(n) = \sum_{i=1}^d \text{count}(A, n, d) \quad \left[\begin{array}{l} \text{Time complexity} \\ \text{of Counting sort} \\ T(n) \in O(n+k) \end{array} \right]$$

$$= \sum_{i=1}^d O(n+k) = O(d(n+k))$$

// Radix sort time complexity

$T(n) \in O(d(n+k)) \rightarrow$ Worst case

$T(n) \in \Omega(d(n+k)) \rightarrow$ Best case

$T(n) \in \Theta(d(n+k)) \rightarrow$ Avg case

// Space complexity

Auxiliary space required Counting Sort
 $(B[] + C[])$

output array $\rightarrow$

Count array $\leftarrow$

Space complexity $S(n) = O(n+k)$

n size of
 o/p $B[]$ array $\rightarrow$

k size of
 Count array $\leftarrow$

$B[1-n]$

$C[0-k]$

BUCKET SORT

- Divide the input elements into several buckets, sort each bucket individually and then combines all bucket to obtain the final sorted array.
- Bucket sort is a *space trade off algorithm* because it uses extra memory (buckets) to reduce sorting time and achieve near linear time complexity.

// Time complexity

Best case : $T(n) \in \Omega(n)$

Avg case : $T(n) \in \Theta(n)$

Worst case : $T(n) \in O(n^2)$

// Space complexity

Bucket sort

$$S(n) \in O(n+k)$$

Number of input insert all bucket. Number bucket

Apply bucket sort input

0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21,
 0.12, 0.23, 0.68

STEP:1

Create n -empty bucket by dividing the interval $[0,1]$ into n equal buckets.

$n=10$ [number input element]

Bucket range $[i/10, (i+1)/10]$

B[0]	[0.0, 0.1]
B[1]	[0.1, 0.2]
B[2]	[0.2, 0.3]
B[3]	[0.3, 0.4]
B[4]	[0.4, 0.5]
B[5]	[0.5, 0.6]
B[6]	[0.6, 0.7]
B[7]	[0.7, 0.8]
B[8]	[0.8, 0.9]
B[9]	[0.9, 1.0]

STEP:2 Insert each element into bucket based on its values and sort by using insertion sort.

$$B[L(n * A[i])]] = A[i]$$

A[]		B[]	
1	0.78	0	
2	0.17	1	
3	0.39	2	
4	0.26	3	
5	0.72	4	
6	0.94	5	
7	0.21	6	
8	0.12	7	
9	0.23	8	
10	0.68	9	

STEP: 3

Concatenate all bucket to obtain the finally sorted array.

0.12, 0.17, 0.21, 0.23, 0.26, 0.39, 0.68, 0.72, 0.78, 0.94 → dist sorted!!!

//ALGORITHM BucketSort(A, n)

BucketSort(A, n)

let create bucket $B[0 \dots n-1]$

for $i=0$ to $n-1$

make $B[i]$ an empty list

for $i=i$ to n

$B[\lfloor n * A[i] \rfloor] = A[i]$

for $i=0$ to $n-1$

Sort list $B[i]$ using insertion sort

Concatenate the list Bucket $B[0], B[1], \dots$

$B[n-1]$ together in order.

//time complexity

$$T(n) = \sum_{i=0}^{n-1} 1 + \sum_{i=1}^n 1 + \sum_{i=0}^{n-1} \text{insertionSort}(l)$$

creating
bucket and initialize
empty list

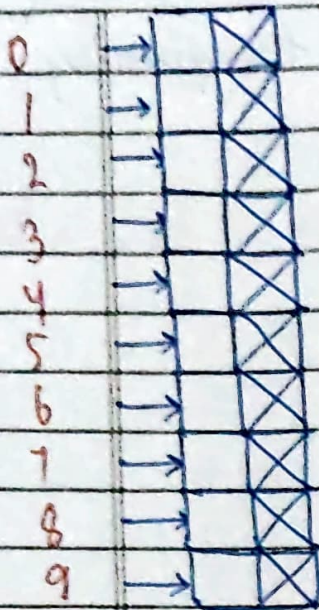
insert $A[i]$
into bucket $B[\lfloor n * A[i] \rfloor]$

Best case:

Input elements are perfectly uniformly distributed
 [Each bucket received at most one element]

Eg: 0.61, 0.37, 0.26, 0.93, 0.41, 0.33, 0.17, 0.01, 0.77, 0.83

M T W T F S S
☐ ☐ ☐ ☐ ☐ ☐ ☐



In this case each bucket has at most 1 element

Time complexity insertion sort $T(i) = O(1)$

$$= [(n-1) - 0 + 1] + (n-i+1) + \sum_{i=0}^{n-1} O(1)$$

$$= n + n + \dots + n$$

$$T(n) = 3n \quad (\text{ignore constant})$$

$$T(n) = \Omega(n) \rightarrow \text{Best Case time complexity.}$$

Worst case:

Input element are not uniformly distributed

[This case all elements go into one bucket]

Eg: 0.45, 0.41, 0.42, 0.48, 0.49, 0.43, 0.44, 0.46, 0.47, 0.40

Sort this Bucket using insertion sort
 time complexity $T(n) \in O(n^2)$

Worst time complexity Bubble Sort depends on

$$T(n) \in O(n^2)$$

Average case $T(n) \in O(n)$

SELECTION PROBLEM

Find the smallest element in an array.

This is called the i th order statistic.

without applying entire array sorting...

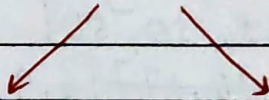
$min = A[0]$

for $i = 1$ to $n-1$

if $(a[i] < min)$

$min = a[i]$

Selection Problem



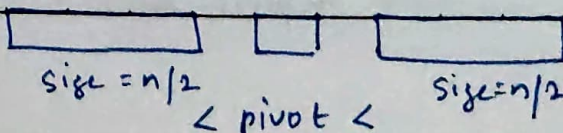
Random Select Algorithm

Selection Algorithm using medians of median

SELECTION ALGORITHM USING MEDIAN OF MEDIAN

- > Select algorithm used to find the smallest element in an unsorted array with guaranteed linear time complexity $O(n)$ (in all cases)
- > Select algorithm chooses a good pivot using medians of median.

$$pivot = medianofmedian()$$



Apply selection Algorithm using medians of median.

45, 12, 3, 19, 8, 23, 7, 2, 50, 30, 17, 06, 11, 9

find 3rd smallest $i=3$.

STEP:1 Divide into group of 5 elements.

$G_1 = [45, 12, 3, 19, 8]$

$G_2 = [23, 7, 2, 50, 30]$

$G_3 = [17, 06, 11, 9]$

STEP:2 Sort each group and find median of each group.

$G_1 = [3, 8, 12, 19, 45]$

$m_1 = 12$

$G_2 = [2, 7, 23, 30, 50]$

$m_2 = 23$

$G_3 = [6, 9, 11, 17]$

$m_3 = 9$

(Choose lower one)

$M = [12, 23, 9]$

STEP:3 Find good pivot by using medians of median

$M = \text{Sort}(M)$

$M = [9, 12, 23]$

12 $\rightarrow$ median of median (M)

pivot = 12

STEP:4 Partition around pivot = 12

0	1	2	3	4	5	6	7	8	9	10	11	12	13
3	8	7	2	6	11	9	12	45	19	23	50	30	17

< pivot <

$$\text{rank} = k = q - \text{low} + 1$$

$$= 7 - 0 + 1 = 8$$

8th minimum element

STEP: 5 Find rank pivot $k = q - \text{low} + 1$
 $= 7 - 0 + 1 = 8$
 pivot $\rightarrow$ 8th min.

STEP: 6 compare i and $k = 8$
 if ($i < k$) search left subarray
 low 0 1 2 3 4 5 6 $q-1$

3	8	7	2	6	11	9
---	---	---	---	---	----	---

STEP: 7 Repeat all process (Steps 1-6)

3	8	7	2	6	11	9
---	---	---	---	---	----	---

$G_{11} =$

3	8	7	2	6
---	---	---	---	---

$G_{12} =$

11	9
----	---

Sort groups G_1 and G_2 find median

$G_{11} =$

2	3	6	7	8
---	---	---	---	---

$m_1 = 6$

$G_{12} =$

9	11
---	----

$m_2 = 9$

STEP: 8 Find good pivot by using medians of medians.

$m = [6, 9]$

$m = \text{sort}(m)$

$m = [6, 9]$

pivot = medians of median(m)

low

pivot = 6

3 4 5 6

[3 | 2]

[6]

[8 | 7 | 11 | 9]

0 1

pivot

Find pivot rank or actual position

$$k = q - \text{low} + 1$$

$$= 2 - 0 + 1$$

$$= 3$$

Compare i and $k = 3$

if ($i == k$)

$3 == 3$

return pivot

3rd smallest element = 6

ALGORITHM

select (A, low, high, i)

if (low == high) return A[low]

Divide A[low...high] into groups of 5 elements each

for each group

sort the group $G_1, G_2, \dots, G_n$

find median of each group and to $m[]$

Let $m[]$ be the array of all medians

$m = \text{sort}(m)$

pivot = medians of median(m)

```

q = partition (A, low, high, pivot)
k = q - low + 1
if (i == k) return A[q]
else if (i < k) return select (A, low, q-1, i)
else return select (A, q+1, high, i-k)
    
```

DISJOINT SETS (Also called union-find)

Disjoint set data structure is used to maintain a collection of non-overlapping sets

$$S_1 = \{ 2, 3, 4, 7 \}$$

$$S_2 = \{ 1, 5, 6, 8, 9 \}$$

$$S_1 \cap S_2 = \emptyset$$

union(7, 6) find $S_1(7) \neq \text{find } S_2(6)$

$$S_3 = \{ 2, 3, 4, 7, 1, 5, 6, 8, 9 \}$$

Basic Disjoint Set operation $\rightarrow$ three main ops

i) Make-set (x)

ii) Find-set (x)

iii) Union (x, y)

Application Disjoint-Set operations

- $\rightarrow$ Find connected components in an undirected graph
- $\rightarrow$ implementing Kruskal algorithm
- $\rightarrow$ Cycle detection in graph
- $\rightarrow$ Cluster problems

makeSet(1), makeSet(2), makeSet(3) ... makeSet(8)

{1}, {2}, {3}, {4}, {5}, {6}, {7}, {8}

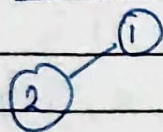
parent[] = [1, 2, 3, 4, 5, 6, 7, 8]

→ union(1, 2)

find(1) ≠ find(2)
 $1 \neq 2$

{1, 2}, {3}, {4}, {5}, {6}, {7}, {8}

parent = [1, ~~2~~, 3, ~~4~~, 5, ~~6~~, 7, ~~8~~]

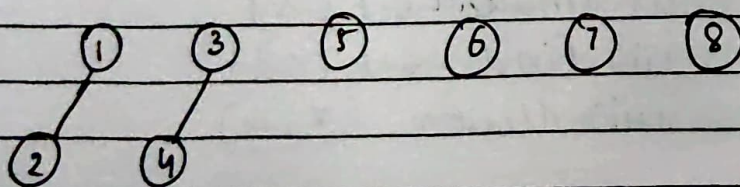


→ union(3, 4)

if (find(3) ≠ find(4))
 union(3, 4)

parent[4] = 3

{1, 2}, {3, 4}, {5}, {6}, {7}, {8}



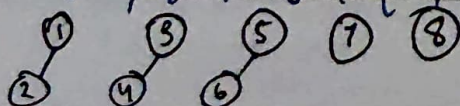
→ union(5, 6)

find(5) ≠ find(6)

union(5, 6)

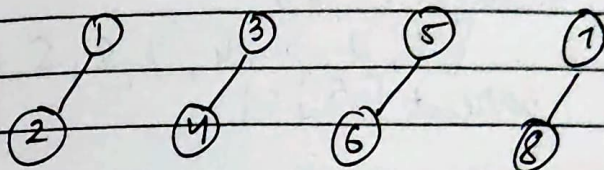
⇒ parent[6] = 5

{1, 2}, {3, 4}, {5, 6}, {7}, {8}

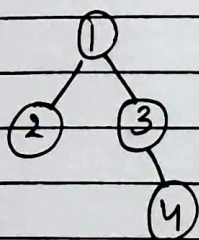


→ union (7, 8)
 find (7) $\neq$ find (8)
 union (7, 8)
 parent [8] = 7

$\{1, 2\}$, $\{3, 4\}$, $\{5, 6\}$, $\{7, 8\}$

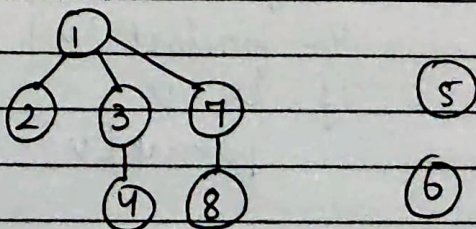


→ union (1, 3)
 find (1) = find(3)
 union (1, 3)
 parent [3] = 1
 $\{1, 2, 3, 4\}$, $\{5, 6\}$, $\{7, 8\}$



→ union (4, 8) root 1 root 7
 find (4) = find (8)

union (4, 8) $\{1, 2, 3, 4, 7, 8\}$,
 parent [7] = 1 $\{5, 6\}$



union(3, 7)

find(3) == find(7)

union(3, 1) not possible

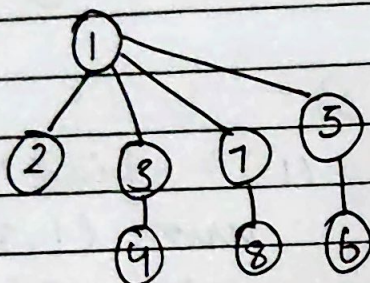
union(4, 6)

find(4) $\neq$ find(6)

union(4, 6)

{1, 2, 3, 4, 7, 8, 5, 6}

parent[5] = 1



union(2, 3) union(4, 6)

ALGORITHM DISJOINT

MakeSet (v)

parent[v] = v

findset (int v)

while (v $\neq$ parent[v])

v = parent[v]

endwhile

return v

UnionSet (x, y)

u = findset(x)

v = findset(y)

if u $\neq$ v

parent[v] = u

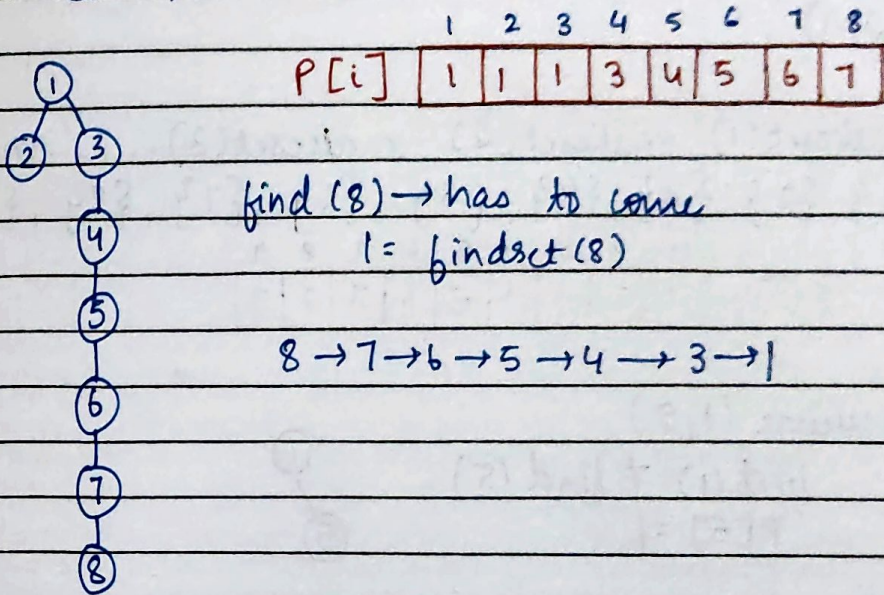
// time complexity

findset(x)

$T(n) \in O(n)$

DISJOINT SET USING PATH COMPRESSION OPTIMIZATION

The optimization is designed for speeding up findset() method (findset() method used to find root parent or leader)



find(8) → has to come
 $l = \text{findset}(8)$

$8 \rightarrow 7 \rightarrow 6 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 1$

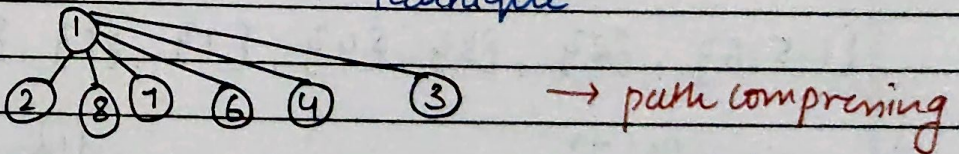
Traditional findset()

// time complexity

$T(n) \in O(n)$

Tree becomes flat

→ using → optimization using path compression technique



→ path compressing

findset() Time Complexity $T(n) = O(1)$

↳ almost constant

findset() using path compression Algorithm

```
findset (int v)
    if (v == parent[v])
        return v
    return parent[v] = findset (parent[v])
```

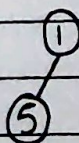
Qn) Find disjoint set union with path compression
 (1,5) (5,6) (6,7) (5,8) (2,3) (3,4) (3,9) (6,8)
 (2,9)

=> makeSet(1) makeSet(2) makeSet(3) ... makeSet(9)
 $\{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{7\}, \{8\}, \{9\}$

	1	2	3	4	5	6	7	8	9
P[]	1	2	3	4	5	6	7	8	9

=> union (1,5)

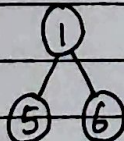
find(1) $\neq$ find(5)
 $P[5] = 1$



$\{1,5\}, \{2\}, \{3\}, \{4\}, \{6\}, \{7\}, \{8\}, \{9\}$

=> union (5,6)

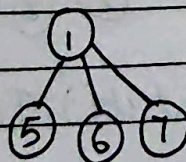
find(5) $\neq$ find(6)
 root of 5 = 1 root of 6 = 6
 $P[6] = 1$



$\{1,5,6\}, \{2\}, \{3\}, \{4\}, \{7\}, \{8\}, \{9\}$

=> union (6,7)

find(6) $\neq$ find(7)
 root = 1 root = 7
 $P[7] = 1$ (not equal)



$\{1, 5, 6, 7\}, \{2\}, \{3\}, \{4\}, \{8\}, \{9\}$

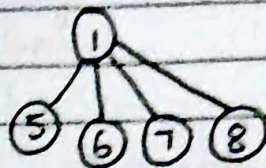
$\Rightarrow$ union (5, 8)

find (5) $\neq$ find (8)

root = 1

root = 8

$p[8] = 1$



$\{1, 5, 6, 7, 8\}, \{2\}, \{3\}, \{4\}, \{9\}$

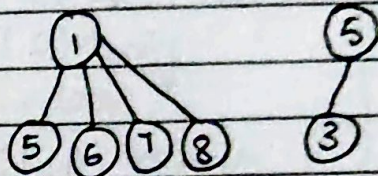
$\Rightarrow$ union (2, 3)

find (2) $\neq$ find (3)

root = 2

root = 3

$p[3] = 2$



$\{1, 5, 6, 7, 8\}, \{2, 3\}, \{4\}, \{9\}$

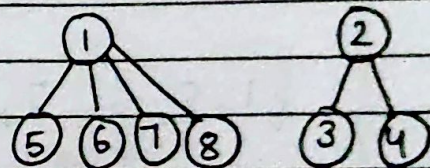
$\Rightarrow$ union (3, 4)

find (3) $\neq$ find (4)

root = 2

root = 4

$p[4] = 2$



$\{1, 5, 6, 7, 8\}, \{2, 3, 4\}, \{9\}$

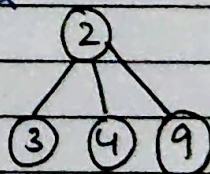
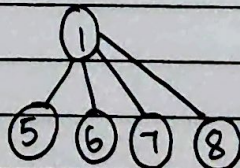
$\Rightarrow$ union (3, 9)

find (3) $\neq$ find (9)

root = 2

root = 9

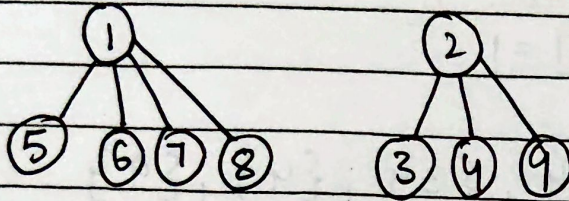
$p[9] = 2$



=> union (6, 8)

find (6) == find (8)
 root = 1 root = 1

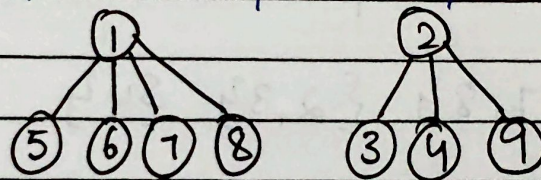
(6, 8) same group union not possible
 [no change]



=> union (2, 9)

find (2) = find (9)
 root = 2 root = 2

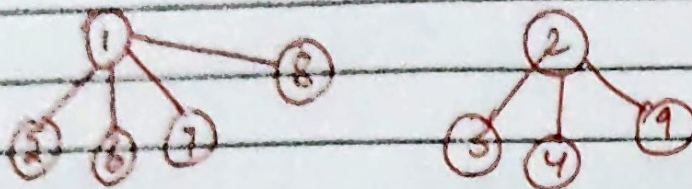
union (2, 9) not possible, belong to same group



{1, 5, 6, 7, 8} , {2, 3, 4, 9}
 Group.1 Group.2

UNION By RANK

- Optimization idea, smaller tree under larger tree
- Denoting who is next parent node



1	2	3	4	5	6	7	8	9
1	2	2	2	1	1	1	1	2

1	2	3	4	5	6	7	8	9
4	3	0	0	0	0	0	0	0

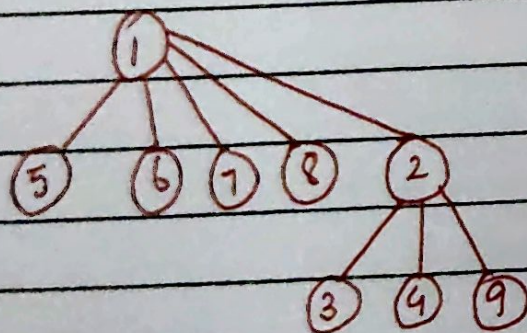
Rank:

$\text{union}(7, 4)$

$$u = \text{find}(7) = 1$$

$$v = \text{find}(4) = 2$$

if $[\text{Rank}[u] > \text{Rank}[v]]$
 $\text{parent}[v] = u$



AMORTIZED ANALYSIS (3 method).

M T W T F S S
☐ ☐ ☐ ☐ ☐ ☐ ☐

COMPASS

Date: _____

union (a, b)

u = find(a)

v = find(b)

if (u != v)

if (rank[u] > rank[v])

parent[v] = u

else if (rank[u] < rank[v])

parent[u] = v

else

parent[v] = u

Rank[i]++

~~Ques~~